

Real-Time Airport Congestion and Traffic Flow Analytics

Valentina Macek

Real-Time Big Data Processing

June 2026

Abstract. *This project implements an end-to-end streaming pipeline that monitors aircraft activity around major airports in the Alpine Europe region in real time. Live aircraft state vectors are ingested from the OpenSky Network REST API and combined with static aircraft metadata. A stream-processing job assigns each aircraft to its nearest airport, classifies its behaviour (inbound, outbound, holding, runway activity), and aggregates per-airport metrics over short time windows, including a composite congestion score. Results are persisted in a relational database and visualised through live dashboards in Grafana. By aggregating live flight data, the system identifies and displays which airports in the region are currently the busiest and how traffic evolves over time.*

Contents

1	Application Domain	2
2	Description of the Data Sources	2
3	Technologies and Overall Architecture	3
3.1	Technologies	3
3.2	System Architecture	4
3.3	Workflow	4
4	Functionalities	5
5	Lessons Learned	10

1 Application Domain

Air traffic around major airports is highly dynamic: the number of aircraft approaching, departing, or circling an airfield can change dramatically within minutes, driven by weather, scheduling peaks, knock-on delays, and congestion that propagates across the wider network. A short burst of arrivals can saturate an airport’s approach airspace, while a single weather event can affect and reshape traffic at all neighbouring airfields. Because this behaviour unfolds on a timescale of seconds to minutes, a static or periodically refreshed view captures little of it; the value lies in observing the activity *as it happens*. Real-time visibility of this kind is useful for situational awareness, for understanding how congestion builds and dissipates across a region, and for comparing how peak periods differ from one airport to another.

This project focuses on real-time monitoring of aircraft activity around a set of airports in the Alpine Europe region. By continuously aggregating live flight data, the system is able to answer questions such as: *Which airports in the region are busiest right now? How is congestion evolving over the last minutes and hours? Which aircraft are currently inbound to a given airport, and are any of them stuck in a holding pattern?* Rather than reporting on a single airport in isolation, the system presents the region as a whole, so that activity at one airfield can be read in the context of its neighbours.

The chosen geographic scope spans Austrian, German, Swiss, Italian, and Slovenian airports (Vienna, Munich, Zurich, the Milan airport cluster, Venice, Ljubljana, and others). This region offers a good mix of large international hubs and smaller regional airports, so the resulting congestion picture is varied and spatially interesting rather than dominated by a single airfield.

The application is best understood as an informative, illustrative tool that shows the real-time congestion situation across the region at a glance. Several audiences could find such a view useful: aviation enthusiasts following live traffic, travellers wanting a sense of how busy their departure or destination airport is on a given day, and anyone interested in spotting anomalies — for instance, noticing that an airport which is normally busy suddenly has most of its inbound aircraft stuck in holding patterns, can be a possible sign of an incident or some other problematic event. It is deliberately an analytical and illustrative tool rather than an operational air-traffic-control system: its classifications are derived from publicly available position data and are best understood as informative estimates rather than exact official numbers.

2 Description of the Data Sources

The system combines one high-velocity streaming source with one static reference source.

Live aircraft state vectors (OpenSky Network REST API). The primary source is the OpenSky Network, which exposes live aircraft *state vectors* through a REST API. Each state vector describes one aircraft at a point in time and includes its identifier (the aircraft’s ICAO 24-bit transponder address, exposed by the connector under the field name `id`), call sign, position (latitude/longitude), barometric altitude, velocity, heading, vertical rate, and an onGround flag. A single query over the project’s bounding box returns the state of every aircraft currently within that region — typically several hundred aircraft over the Alpine Europe region at a busy time of day. The source is characterised by:

- **Velocity:** the data is genuinely live and is polled on a fixed interval, configurable via the `.env` file (set to 60 seconds in our final configuration). Each poll yields a fresh snapshot of the airspace.
- **Volume:** hundreds of records per poll, continuously, producing a steady stream into the messaging layer.

- **Variety/complexity:** the payload is JSON with `camelCase` field names produced by the connector, and many fields are nullable (an aircraft may not report altitude, call sign, or vertical rate), which the processing logic must tolerate.

Access is anonymous — the connector does not support OpenSky’s current authentication scheme — which imposes a usage budget: requests are rate-limited and charged in proportion to the queried area, a constraint that shaped several design decisions (Section 5).

Static aircraft metadata (OpenSky aircraft database). The second source is an extract of the OpenSky aircraft database, downloadable `registrations.csv` file. It contains four columns per aircraft: `icao24`, `typecode` (aircraft type code,¹ e.g. BE36), `operatoricao` (operator code²), and `built` (year built). Unlike the state vectors, this is not a stream: the file is large (on the order of half a million aircraft) but static and very slow-changing, so it is treated as reference data and loaded once into a PostgreSQL lookup table. Its `icao24` column holds the same ICAO 24-bit transponder address that the state vectors expose under the field name `id`; although the two sources label this identifier differently, it is a common key, and the dashboards join the live aircraft detail against this lookup table on `icao24` at query time to add aircraft metadata available.

3 Technologies and Overall Architecture

3.1 Technologies

Docker & Docker Compose

The entire system runs as a set of containers orchestrated by a single `docker-compose.yml`. Each component — broker, connectors, processing cluster, database, and dashboards — runs in its own container, which keeps them isolated and independently restartable. Compose manages the dependencies between them, using health checks so that a service only starts once the services it relies on are actually ready (for example, the processing job waits for Kafka and the database to be healthy). This makes the whole stack reproducible and startable with a single command.

Apache Kafka (with ZooKeeper)

Apache Kafka is the central message broker that decouples ingestion from processing: the OpenSky connector publishes the live aircraft positions into the `state_vectors` topic and the Flink job consumes from it, so the two sides operate independently and a slow or restarting consumer never blocks ingestion. ZooKeeper provides the coordination and metadata management the broker depends on.

Kafka Connect

Kafka Connect performs the ingestion declaratively, through a configured connector, so no custom producer code has to be written or maintained. An OpenSky source connector polls the OpenSky REST API for aircraft positions and publishes them to `state_vectors`; the polling interval (60 seconds in our final configuration) and the geographic bounding box are both set in the `.env` file. The connector is defined entirely in a JSON configuration file. (The static aircraft metadata is *not* ingested through Kafka — being a large, static file, it is loaded directly into PostgreSQL instead; see Section 5.)

Apache Flink / PyFlink

Apache Flink is the stream-processing engine at the heart of the system; it was chosen because it is a true streaming framework with first-class support for stateful processing and time-based windowing, both of which this project relies on. The job is written in Python using the PyFlink

¹https://en.wikipedia.org/wiki/List_of_aircraft_type_designators

²https://en.wikipedia.org/wiki/List_of_airline_codes

DataStream API and runs on a Flink *session cluster*: a JobManager coordinates and schedules work, a TaskManager executes the actual operators, and a lightweight client container submits the job to the cluster. The job consumes the live state-vector stream and, for each aircraft, parses the record, assigns it to its nearest airport using the Haversine distance, and classifies its behaviour (inbound, outbound, arriving, departing, holding, runway load). The classified stream is then keyed by airport and aggregated over one-minute tumbling windows to produce the per-airport metrics, and the same stream is fanned out to additional branches that maintain the per-aircraft inbound and holding detail. Flink’s keyed state and windowing are what make these per-airport aggregations possible within a single continuous job.

PostgreSQL & pgAdmin

PostgreSQL is the relational store that sits between the processing job and the dashboards. A relational database was a natural fit because the output of the job is well-structured tabular data that Grafana queries with SQL. The schema separates concerns across several tables: a static `airports` reference table; `airport_metrics`, holding one aggregated row per airport per time window; `congestion_history`, a time series feeding the “over time” panel; and `inbound_aircraft` and `holding_aircraft`, holding live per-aircraft detail. Foreign keys from the metric tables back to `airports` enforce referential integrity. Writes from Flink are made *idempotent* using `INSERT ...ON CONFLICT ...DO UPDATE` (upserts): re-processing the same window or aircraft updates the existing row rather than creating duplicates, which keeps the tables consistent even if the job restarts. The per-aircraft tables are upserted on every cycle and carry a `last_seen` timestamp so that stale aircraft can be filtered out, and the holding table tracks each aircraft’s `holding_since` time so its holding duration accumulates in the database across cycles. A separate `aircraft_metadata` table holds the static aircraft information (aircraft type code and operator) loaded from the OpenSky aircraft database; rather than joining this large reference set inside the streaming job, the dashboards join it against the live aircraft detail at query time on `icao24`, which keeps the streaming job small and pushes the lookup into the database, where joining a large static table is cheap. pgAdmin provides a browser-based UI for inspecting the tables and running ad-hoc queries during development.

Grafana

Grafana provides the live dashboards that visualise the data. It connects to PostgreSQL as a data source and renders a range of panel types — bar gauges, bar and time-series charts, tables, and a geographic map — each driven by an SQL query against the metric tables, and refreshes automatically so the views stay current. Both the data source and the dashboard are *provisioned* from version-controlled configuration files rather than configured by hand in the UI, so the visualisation comes up automatically with the rest of the stack and is fully reproducible.

Python libraries

`psycpg2` for the PostgreSQL sink and `confluent-kafka` for a small utility that waits for the input topics before the job starts.

3.2 System Architecture

Figure 1 shows the overall data flow. The live state vectors are ingested through Kafka and processed by a PyFlink job that aggregates them and writes the results to PostgreSQL; the static aircraft metadata is loaded into a PostgreSQL lookup table. Grafana then queries PostgreSQL, joining the live aircraft data with the metadata at query time, to render the dashboards.

3.3 Workflow

Setup (manual, once). The user configures a `.env` file (database credentials, Grafana credentials, and the OpenSky bounding box that defines the monitored region). A PowerShell script

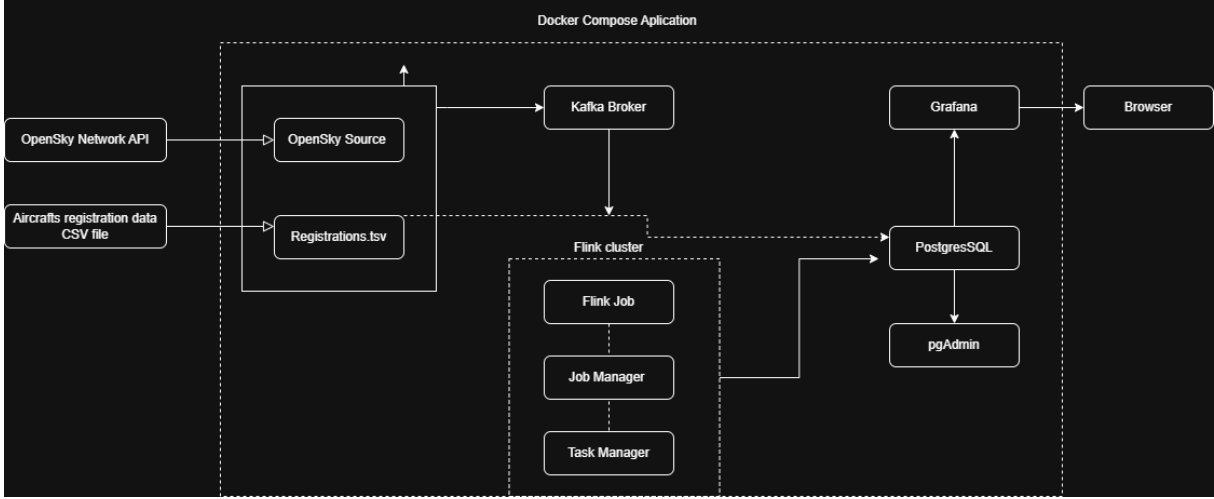


Figure 1: System architecture

optionally prepares a local Python virtual environment for development. The whole stack is then started with a single command:

```
docker compose up -d
```

On start-up, Compose brings the services up in dependency order; an init container creates the Kafka topic, and an entrypoint registers the OpenSky source connector. The PyFlink job is submitted to the Flink session cluster. On its first start, PostgreSQL also runs initialisation scripts that create the schema and automatically load the static aircraft metadata from the supplied file into its lookup table, so no manual data-loading step is required.

Data processing (automatic). For each incoming state vector the PyFlink job:

1. parses the JSON and discards records without a position;
2. assigns the aircraft to its nearest airport using the Haversine distance, keeping only aircraft within a configurable radius (50 km);
3. classifies the aircraft as inbound, outbound, arriving, departing, holding, and/or contributing to runway load, based on altitude, vertical rate, and the on-ground flag;
4. aggregates aircraft per airport over a one-minute tumbling window and computes per-airport counts plus a composite congestion score;
5. writes the aggregates to `airport_metrics` and `congestion_history`, and upserts per-aircraft detail into `inbound_aircraft` and `holding_aircraft`.

Grafana queries PostgreSQL on a refresh interval, joining the per-aircraft detail against the static `aircraft_metadata` table on `icao24` to add aircraft type and operator, and renders the live dashboards.

Manual actions during operation. Because OpenSky’s anonymous access is rate-limited, the operator may pause the polling connector to let the usage budget recover, and tune the bounding box and polling interval in `.env` to stay within the limit. These are the only routine manual interventions.

4 Functionalities

The dashboard presents, for each airport, a set of complementary views. All panels are driven by the tables populated by the PyFlink job and refresh automatically.

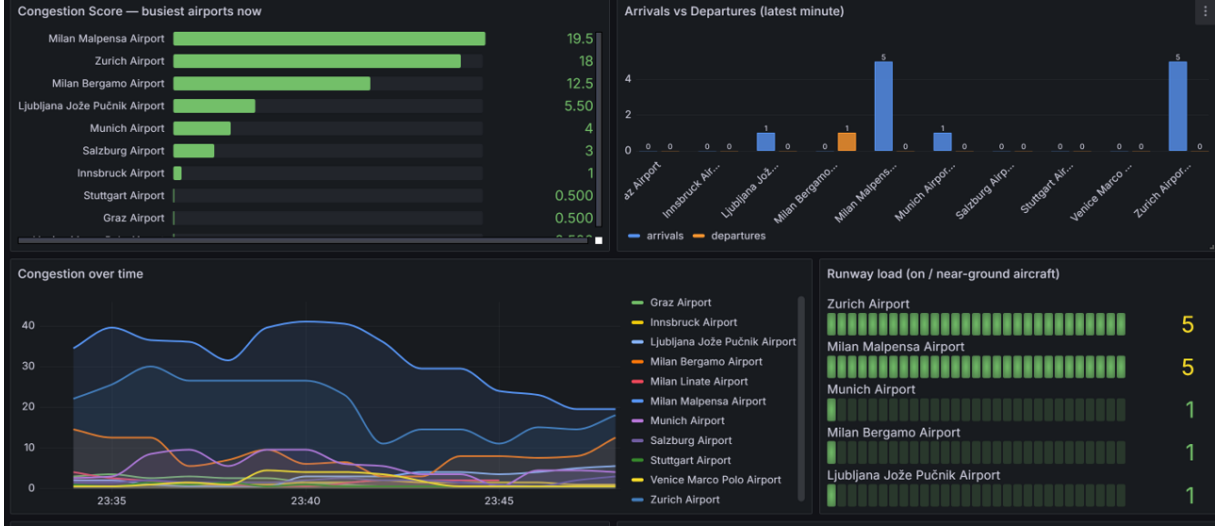


Figure 2: First part of the live dashboard



Figure 3: Congestion score per airport (busiest first).

Congestion score. A real-time congestion index per airport, shown as a ranked bar gauge so the busiest airports are immediately visible. The score is a weighted combination of inbound and outbound aircraft, holding aircraft, runway load, and overall aircraft density in range, computed per window as

$$score = 2.0 n_{in} + 1.5 n_{out} + 2.5 n_{hold} + 1.0 n_{runway} + 0.5 n_{total},$$

where n_{in} , n_{out} , and n_{hold} are the counts of inbound, outbound, and holding aircraft in the window, n_{runway} is the runway-load count, and n_{total} is the total number of distinct aircraft in range of the airport. The weights reflect how much each factor contributes to perceived congestion: holding and inbound traffic are weighted most heavily, since aircraft converging on or stacked above an airport are the clearest signs of pressure, while overall density contributes a small baseline term. The weights are configurable and were chosen empirically rather than derived from operational data. Colour thresholds highlight when an airport crosses from quiet to busy.

Arrivals and departures. A grouped bar chart of live arrival and departure counts per airport for the most recent minute, letting the relative balance of traffic at each airport be compared at a glance.



Figure 4: Arrivals versus departures (latest window).

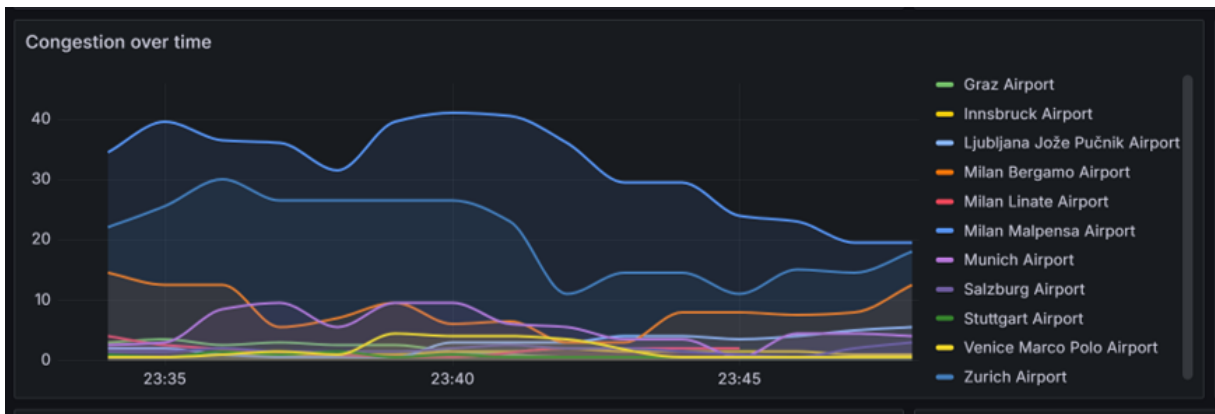


Figure 5: Congestion evolution over time.

Congestion over time. A multivariate time series showing how each airport’s congestion score evolves, so trends and peaks over the last minutes and hours are visible rather than just the value this exact minute.

Runway load. A gauge approximating runway load, based on aircraft that are on the ground or very low near the airport — a proxy for how busy the runway and its immediate surroundings are.

Inbound aircraft overview. A table listing individual inbound aircraft with detailed information: airport, call sign, aircraft type and operator (from the registration join), altitude, speed, and distance to the airport. Only aircraft seen in the last two minutes are shown, so the table reflects the current situation.

Holding pattern detection. A table of aircraft currently estimated to be in a holding pattern — loitering near an airport at low altitude without a clear climb or descent — together with how long each has been holding. The holding duration is tracked in the database so that it accumulates across processing cycles.

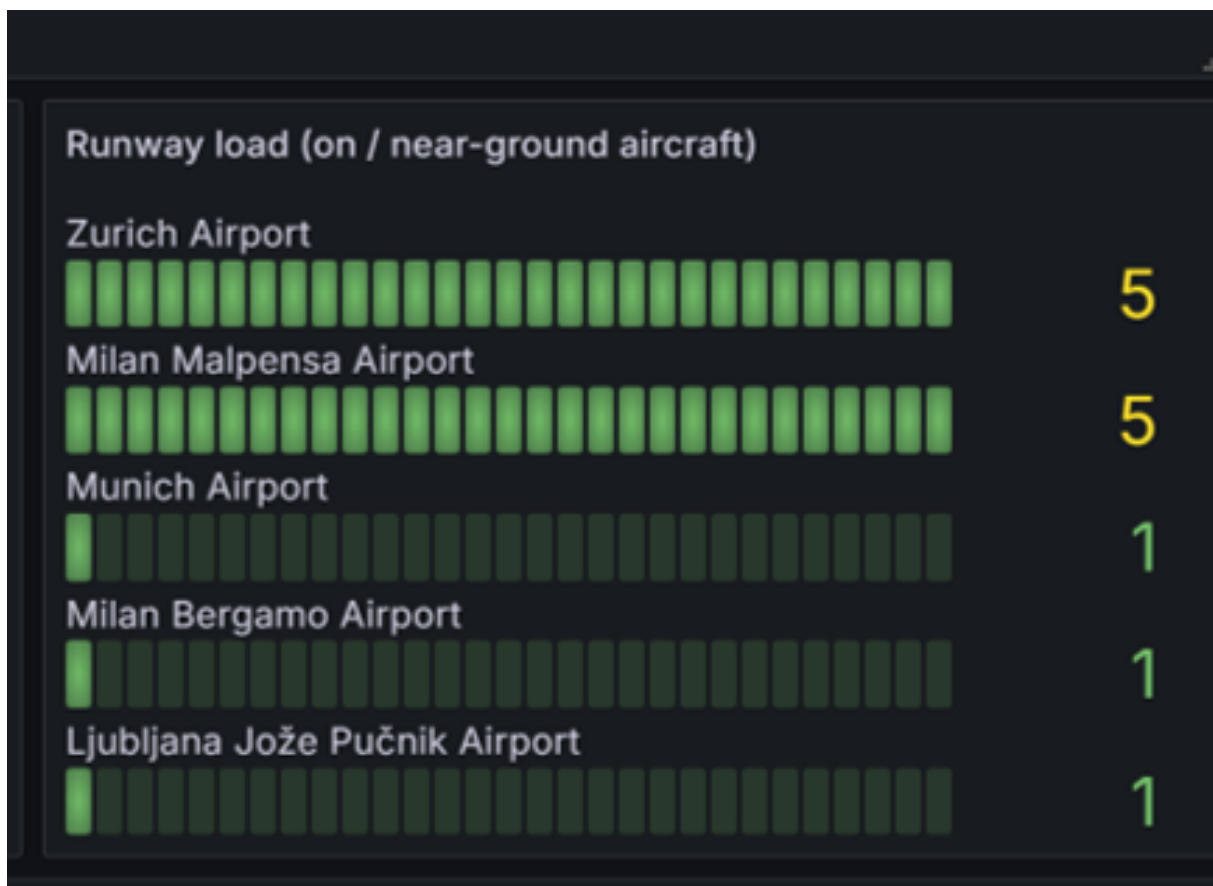


Figure 6: Runway load indicator.

airport ▼	callsign ▼	type ▼	operator ▼	icao24 ▼
Graz Airport	DLA66U			300a7c
Innsbruck Airport	OEGTR	C56X		440c9b
Ljubljana Jože Pučnik	BCY041P	CRJ9		4cacbd
Milan Bergamo Airport	MABTE			424c98
Milan Bergamo Airport	AUA521S	E195	AUA	440821
Milan Bergamo Airport	ICNTK			3009fb
Milan Bergamo Airport	ICNTX			3009fc

Figure 7: Inbound aircraft overview table.

Holding Pattern Detection				
airport 	callsign 	type 	operator 	icao24 
Munich Airport	N8889	FA7X		ac3f20
Salzburg Airport	OEDST	DA40		440b5d

Figure 8: Holding pattern detection with duration.

Regional heatmap. A geographic map placing each airport at its real location as a marker whose size and colour reflect its current congestion, so the busiest airports in the region can be identified instantly. Because the map is driven by live data, it also captures how traffic shifts over the course of a day. Figure 11 contrasts a quieter period with a busy one: the congestion scale and the markers grow significantly between the two, with Milan and Zurich standing out in otherwise quiet night time and Zurich dominating busy morning hour.

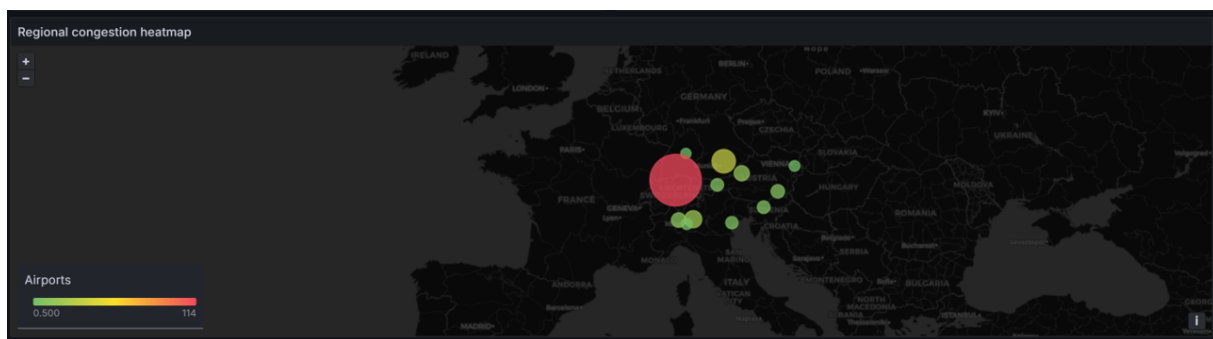


Figure 9: Busy period (congestion scale up to ≈ 114).



Figure 10: Quieter period (congestion scale up to ≈ 20).

Figure 11: Regional congestion heatmap at two times of day. Marker size and colour scale with each airport's congestion; both the colour scale (bottom-left legend) and the marker sizes grow substantially from the quieter period to the busy one.

5 Lessons Learned

What worked well. The container-based, configuration-as-code approach paid off repeatedly. Defining the whole stack in Compose, and provisioning both the database schema and the Grafana data source and dashboard from files, meant the system could be torn down and rebuilt deterministically, and that the dashboard definitions lived in version control rather than only inside Grafana’s internal database. Decoupling ingestion from processing via Kafka let each stage be debugged independently: it was always possible to ask “*is data in the topic?*” separately from “*is Flink processing it?*”. Keeping the streaming job focused on the live data, and pushing the static-metadata lookup into PostgreSQL as a query-time join, kept the job small and fast; and tracking holding *duration* in the database (rather than in Flink state) made that feature simple and robust.

What did not work, or was harder than expected. The most significant external obstacle was data access. OpenSky discontinued username/password (basic) authentication, moving to an OAuth2 token flow that the chosen Kafka Connect source connector does not support; as a result the system runs against anonymous access. Anonymous access is rate-limited by a usage budget charged in proportion to the queried area, so a large bounding box combined with frequent polling repeatedly exhausted the budget and returned `429 Too Many Requests`, halting ingestion. Tuning the bounding box was a balancing act: the initial box covered a broad swathe of Central Europe and was simply too expensive per request, so it was progressively shrunk to the Alpine region and the polling interval lengthened, trading geographic coverage for the ability to keep running. Even so, the budget remained the binding constraint, and the practical remedy was largely operational — pausing the connector to let the budget recover and bringing it up only for short live windows — rather than something that could be fully solved in code.

A second design lesson came from the metadata enrichment. The original approach joined the two topics inside Flink: the live state vectors and the static registrations were keyed by `icao24` and combined with a stateful keyed join. This worked in principle but did not scale — the OpenSky aircraft database contains on the order of half a million rows, and streaming that entire static table through a keyed join overwhelmed the job, which stalled and stopped emitting results while consuming records. The fix was to stop treating static reference data as a stream: the registrations stream (the FilePulse connector and its Kafka topic) was removed entirely, and the metadata is instead loaded once, directly from the file into a PostgreSQL lookup table, and joined against the live aircraft detail at query time by Grafana. The lesson was that streaming joins suit two live streams, but for a large, slow-changing lookup it is easier to treat it as static and load it independently in the database.

Several further issues were internal and instructive. Early on, running PyFlink directly on a local Windows machine proved impractical — the Python, Java, and Flink dependencies were difficult to align and produced repeated installation and version conflicts. Rather than fight the local environment, we moved the Flink job into a Docker container from the outset, which made the dependencies reproducible and the same on every machine; in hindsight this was the right call and saved considerable time. A silent failure later occurred when the processing job and the database schema drifted apart — the job wrote a column the table did not have, so every insert failed while the rest of the pipeline appeared healthy and the dashboard simply froze on old data. The lesson was to inspect the processing logs (where the database error was plainly reported) rather than assuming the problem was upstream. We also learned that restarting the stack with `start` skipped the health-gated ordering that `up` applies, causing components to launch before their dependencies were ready; using `up` consistently avoided the race. Finally, the registration metadata available to us contained only four columns, so the inbound table shows aircraft *type* and *operator code* rather than full registration and manufacturer — a reminder that enrichment is only as rich as the reference data.

What we would improve. Given more time, the priorities would be: (1) add a small token-fetching component so the pipeline could use OpenSky’s OAuth2 flow and benefit from higher, authenticated rate limits; (2) reimplement holding detection as a proper stateful Flink operator with timers and a circling heuristic, rather than the single-snapshot approximation used here, to reduce false positives; (3) move from processing-time to event-time windows with watermarks for more accurate aggregation under delays; and (4) add a periodic cleanup of the per-aircraft detail tables and basic alerting when an airport’s congestion crosses a threshold.